

# UT1. Características del lenguaje PHP

---

DESARROLLO WEB EN ENTORNO SERVIDOR (DWS)



# Contenidos

---

1. Características generales de PHP
  1. Embebido en HTML
  2. Variables, Cadena de Texto, Tipos de datos, Variables especiales de PHP
2. Estructuras de control
  1. Sentencias condicionales: if, switch
  2. Sentencias de bucles: while, do-while, for (foreach)
3. Tipos de datos compuestos
4. Funciones
5. Formularios web

# Características generales de PHP. Embebido en HTML.

---

PHP (Hypertext Preprocessor): Lenguaje script Open Source que se ejecuta en el servidor, especialmente diseñado para el desarrollo de aplicaciones web que puede ser integrado fácilmente en código HTML.

```
<!DOCTYPE html>
<html>
<head>
<title>Ejemplo</title>
</head>
<body>

<?php
echo "Hola, soy un script PHP!";
?>

</body>
</html>
```

**<?php**

...

**?>**



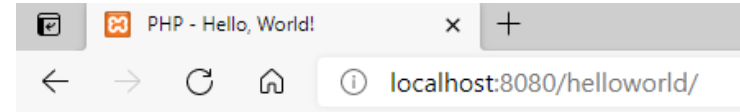
# Características generales de PHP. Embebido en HTML.

## Estructura básica

Lenguaje html

```
<html>  
  <head>  
  </head>  
  <body>  
  </body>  
</html>
```

```
p.php  
<?php  
echo "hello world". "\t";  
echo "\n";  
?>
```



**Hello, World!**



# Características generales de PHP. Embebido en HTML.

---

```
<!DOCTYPE html>
<html>
  <head>
    <title>Prueba PHP</title>
    <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
  </head>
  <body>
    Prueba de página PHP </br>
    El nombre del servidor es: <?php echo $_SERVER['SERVER_NAME']; ?>
  </body>
</html>
```



# Características generales de PHP.

## Variables.

---

Una de las características principales de PHP es que es un lenguaje no fuertemente tipado. Esto es, no es necesario indicar el tipo de dato al declarar una variable. Las variables no se declaran, se crean la primera vez que se les asigna un valor y el tipo de dato depende del valor con que se inicialicen.

En PHP los identificadores de las variables van siempre precedidos por el carácter '\$'. El identificador de la variable debe comenzar por una letra o un guion bajo ('\_') y puede estar formado por números, letras y guiones bajos.

`$nombre = valor;`

Ejemplo: `$entero = 4;`



# Características generales de PHP. Variables.

---

En principio , la asignación de variables se realiza mediante copia. Es decir:

$$\text{\$a} = \text{\$b};$$

Se crea una nueva variable 'a' y se le asigna el valor que tiene 'b'.

También es posible definir una referencia a una variable utilizando el símbolo '&'. Es decir:

$$\text{\$a} = \&\text{\$b};$$

En este caso la variable 'a' no es una nueva variable con el valor de 'b', por el contrario, 'a' apunta al valor de 'b', y si es cambiado el valor de 'a' cambia el valor de 'b' y viceversa.



# Características generales de PHP. Variables.

---

```
$entero = 4;           // tipo integer
$numero = 4.5;         // tipo coma flotante
$cadena = "cadena";    // tipo cadena de caracteres
$bool = True;          //tipo booleano
/* cambio de tipo de una variable */
$a = 5;                 // entero
echo gettype($a);       // imprime el tipo de dato de a
echo "<br>";
$a = "Hola";             // cambia a cadena
echo gettype($a);       // se comprueba que ha cambiado (de integer a string)
```



# Características generales de PHP.

## Variables.

---

El ámbito de una variable es la parte del código en que está es visible. Una variable declarada en un fichero PHP está disponible en ese fichero y en los ficheros que se incluyan desde éste.

Las funciones definen variables locales, de manera que las variables declaradas en una función sólo son accedidas desde la propia función. Desde la función no se puede acceder a otras variables que no sean las locales o sus argumentos.

La definición de variables globales, son accesibles desde cualquier función o fichero de la aplicación y se puede realizar:

1. Usando la palabra reservada *global*.
2. Usando la variable predefinida `$_GLOBALS`.



# Características generales de PHP.

## Variables Especiales.

---

En PHP hay muchas variables predefinidas disponibles. Estas contienen información sobre el servidor, datos enviados por el cliente o variables de entorno.

Dentro de las variables predefinidas hay un grupo denominado *SuperGlobales* que están disponibles desde cualquier ámbito. Cada una de ellas guarda información de un tipo, por ejemplo **\$\_SERVER**.

```
echo "Ruta donde se encuentra la página ejecutándose: " . $_SERVER['PHP_SELF'];
```

```
echo "Nombre del servidor: " . $_SERVER['SERVER_NAME'];
```

```
echo "Software del servidor: " . $_SERVER['SERVER_SOFTWARE'];
```

```
echo "Protocolo: " . $_SERVER['SERVER_PROTOCOL'];
```

```
echo "Método de la petición: " . $_SERVER['REQUEST_METHOD'];
```



# Características generales de PHP.

## Tipos de datos escalares.

---

PHP ofrece cuatro tipos de datos escalares: integer, float, boolean y string.

```
$a = 0b100;      // en binario
$a = 0100;       // octal
$a = 0x100;      // hexadecimal
$a = 3/2;        // la división entre enteros no da problemas
echo $a.'<br>';    // 1.5
$b = 7.5;
$a = (int) $b;    // casting a int
echo $a.'<br>';    // 7, se trunca
$b = 7e2;        // notación científica
$b = 7E2;
```



# Características generales de PHP.

## Cadenas.

---

El tipo de dato string permite almacenar cadenas de caracteres. Para delimitar una cadena es posible utilizar comillas simples o dobles, pero hay una diferencia. En caso de utilizar comillas dobles (“...”) las variables que aparezcan dentro de la cadena se sustituirán por su valor.

```
$var = "Paco";  
  
echo "Hola $var <br>";           // Hola Paco  
  
echo 'Hola $var <br>';           // Hola $var  
  
echo "Hola " . $var;             // Hola Paco
```



# Características generales de PHP.

## Constantes.

---

Para definir constantes se utiliza la función `define()`, que recibe el nombre de la constante y el valor que queremos darle:

```
define("LIMITE", 10000);
```

Por convenio, los identificadores para las constantes se ponen en MAYÚSCULAS.



# Características generales de PHP.

## Comentarios.

---

En PHP se pueden utilizar comentarios:

1. De bloque, encerrados entre “/\*” y “\*/”.
2. De línea, comenzando por “//” o por “#”.

//comentario de línea

\$a = 0; // comentario de línea

/\* comentario

de bloque

\*/

# comentario de una sola línea



# Características generales de PHP.

## Operadores.

---

PHP cuenta con los operadores habituales para operaciones aritméticas, lógicas, de manipulación de cadenas, etc.

Caben destacar entre los operadores de comparación los siguientes “===” y “!==”, llamadas comúnmente *Idéntico* y *No Idéntico* que no están presentes en otros lenguajes.

El operador “===” se usa para comparar dos expresiones y se evalúa como verdadero cuando las dos expresiones tienen el mismo valor y tipo de dato.

Se diferencia del operador “==”, en que las expresiones cuando no tienen el mismo tipo de datos, las intenta convertir antes de comparar.



# Características generales de PHP. Operadores.

---

```
$a = "3";  
$b = 3;  
if ($a == $b) {  
    echo "Son iguales <br>";           // Muestra son iguales  
} else {  
    echo "No son iguales <br>";  
}  
if ($a === $b) {  
    echo "Son idénticos <br>";  
} else {  
    echo "No son idénticos <br>";     // No son idénticos  
}
```



# Estructuras de control.

## Estructuras condicionales.

---

PHP cuenta con las estructuras de control habituales en la programación estructurada para realizar sentencias condicionales y de repetición. Su sintaxis es muy parecida a Java o C.

```
$var = 3;  
if($var < 0) echo "Es menor que cero";  
if ($var > 0){  
    echo "Es mayor que cero";  
}
```



# Estructuras de control.

## Estructuras condicionales.

---

PHP cuenta con las estructuras de control habituales en la programación estructurada para realizar sentencias condicionales y de repetición. Su sintaxis es muy parecida a Java o C.

```
$var = 3;
if ($var < 0) {
    echo "Es menor que cero";
} else {
    echo "Es mayor o igual que cero";
}
```



# Estructuras de control.

## Estructuras condicionales.

---

PHP cuenta con las estructuras de control habituales en la programación estructurada para realizar sentencias condicionales y de repetición. Su sintaxis es muy parecida a Java o C.

```
$var = 3;
if ($var == 1) {
    echo "Es un uno";
} elseif ($var == 2) {
    echo "Es un dos";
} elseif ($var == 3) {
    echo "Es un tres";
} else{
    echo "No es un uno, ni un dos, ni un tres";
}
```



# Estructuras de control.

## Estructuras condicionales.

---

PHP cuenta con las estructuras de control habituales en la programación estructurada para realizar sentencias condicionales y de repetición. Su sintaxis es muy parecida a Java o C.

```
$var = 3;
switch ($var) {
case 1:  echo "Es un uno"; break;
case 2:  echo "Es un dos"; break;
case 3:  echo "Es un tres"; break;
default: echo "No es un uno, ni un dos, ni un tres";
}
```



# Estructuras de control.

## Estructuras de repetición.

---

Las estructuras de repetición o bucles sirven para repetir un conjunto de instrucciones mientras se da una condición. Estas estructuras son: for, while y do-while que tienen la misma sintaxis que en Java o C.

```
for ($i = 0; $i < 5; $i = $i + 1) {  
    echo "$i <br>";  
}
```



# Estructuras de control.

## Estructuras de repetición.

---

Las estructuras de repetición o bucles sirven para repetir un conjunto de instrucciones mientras se da una condición. Estas estructuras son: for, while y do-while que tienen la misma sintaxis que en Java o C.

```
$i = 0;
while ($i < 5) {
    echo "$i <br>";
    $i = $i + 1;
}
```



# Estructuras de control.

## Estructuras de repetición.

---

Las estructuras de repetición o bucles sirven para repetir un conjunto de instrucciones mientras se da una condición. Estas estructuras son: for, while y do-while que tienen la misma sintaxis que en Java o C.

```
$i = 0;  
do {  
    echo "$i <br>";  
    $i = $i + 1;  
} while ($i < 5);
```



# Estructuras de control.

## Estructuras de repetición.

---

Diferencia entre el bucle while y el do-while

```
$i = 0;  
do {  
    echo "En el do-while: $i <br>";  
    $i = $i + 1;  
} while ($i < 0);
```

```
while ($i < 0) {  
    echo "En el while: $i <br>";  
    $i = $i + 1;  
}
```



# Estructuras de control.

## Estructuras de repetición.

---

Dentro de un bucle es posible utilizar las sentencias break y continue. La primera sirve para salir del bucle o switch y la segunda para forzar una nueva iteración del bucle.

```
$i = 0;
while ($i < 5){
    echo "$i <br>";
    $i++; // es lo mismo que $i = $ i + 1;
    if ($i == 3){
        break;
    }
}
```



# Estructuras de control.

## Estructuras de repetición.

---

Dentro de un bucle es posible utilizar las sentencias break y continue. La primera sirve para salir del bucle o switch y la segunda para forzar una nueva iteración del bucle.

```
for ($i = 0; $i < 5; $i = $i + 1) {  
    if ($i == 3) {  
        continue;  
    }  
    echo "$i <br>";  
}
```



# Estructuras de control.

## Estructuras de repetición.

---

Dentro de un bucle es posible utilizar las sentencias break y continue. La primera sirve para salir del bucle o switch y la segunda para forzar una nueva iteración del bucle.

```
for ($i = 0; $i < 5; $i = $i + 1) {  
    if ($i == 3) {  
        continue;  
    }  
    echo "$i <br>";  
}
```



# Estructuras de control.

## Estructuras de repetición.

---

**Ejercicio.** Escribir un programa que calcule el factorial de un número. Recordar que el factorial sólo está definido para números enteros mayores o iguales que cero.

$$5! = 5 * 4 * 3 * 2 * 1$$

$$0! = 1$$



# Estructuras de control.

## Otras estructuras.

---

Para incluir otros ficheros PHP proporciona 4 instrucciones:

1. `include "<NombreFichero>";` // Devuelve un WARNING en caso de que no exista
2. `include_once "<NombreFichero>";` // Sólo se incluye sino ha sido especificado
3. `require "<NombreFichero>";` // Devuelve un ERROR en caso de que no exista
4. `require_once "<NombreFichero>";` // Sólo se requiere sino ha sido ya especificado

Por ejemplo:

```
include "miFichero.php";
```

```
require "miFichero.php";
```



# Tipos de datos compuestos.

## Arrays.

---

En PHP son una estructura muy flexible y potente. Unifica en un solo tipo lo que en otros lenguajes se consigue con arrays básicos, vectores, listas, diccionarios.

1. Los elementos de un array se identifican por una clave, que puede ser un entero o una cadena.
2. Los elementos guardan un orden dentro del array.
3. Este orden está determinado por el orden de los elementos al declarar el array o al añadir nuevos elementos.



# Tipos de datos compuestos.

## Arrays.

---

Los arrays se declaran con la función `array ()` o utilización de la notación de los corchetes `[]` (más utilizada). Por ejemplo:

```
$arr1 = [  
    0 => 444,  
    1 => 222,  
    2 => 333,  
];  
print_r($arr1);  
  
echo "<br>" . "pos 0: " . $arr1[0] .  
"<br>";
```

```
$arr1[0] = 555;  
print_r($arr1);  
echo "<br>";
```

```
$arr2 = array (  
    "1111A" => "Juan Vera Ochoa",  
    "1112A" => "Maria Mesa  
Cabeza",  
    "1113A" => "Ana Puertas Peral"  
);  
$arr2["1113A"] = "Ana Puertas  
Segundo";
```

`print_r($arr1)`: Muestra información sobre la variable que se le pasa como argumento, en un array muestra las claves y los valores de todos los elementos.



# Tipos de datos compuestos.

## Arrays.

---

Para recorrer un array lo habitual es usar el bucle **foreach**.

El cuerpo del bucle se ejecuta una vez por cada elemento del array ( en el ejemplo anterior \$arr). En cada iteración el bucle \$valor almacena el elemento correspondiente. También se puede utilizar especificando una variable para la clave.

```
$arr2 = array (  
    "1111A" => "Juan Vera Ochoa",  
    "1112A" => "Maria Mesa Cabeza",  
    "1113A" => "Ana Puertas Peral"  
);
```

```
foreach ($arr2 as $nombre) {  
    echo "$nombre <br>";  
}  
  
foreach ($arr2 as $codigo => $nombre) {  
    echo "Código: $codigo Nombre: $nombre <br>";  
}
```



# Tipos de datos compuestos.

## Arrays.

---

**Ejercicio 01.** Crea un array con cinco nombres de personas y recórrelo mostrando un mensaje de “Yo soy amigo de <NombrePersona>” .

**Ejercicio 02.** Crea un array con diez números positivos o negativos, y recorre el array sumando el número de elementos positivos, negativos o con valor cero.

**Ejercicio 03.** Almacena en un array los “n” primeros números pares mostrando el resultado.



# Tipos de datos compuestos.

## Arrays.

---

Si se desea modificar los valores de un array al recorrerlo con un foreach, hay que utilizar una referencia al declarar las variables del bucle.

```
$arr1 = array(
    "Viernes" => 22,
    "Sábado" => 34
);
/* no modifica el array */
foreach ($arr1 as $cantidad) {
    $cantidad = $cantidad * 2;
}
print_r($arr1);
```

```
/* modifica el array */
foreach ($arr1 as &$cantidad) {
    $cantidad = $cantidad * 2;
}
print_r($arr1);
```



# Tipos de datos compuestos.

## Arrays.

---

- Si se declara un array sin usar claves, se asignan claves por defecto enteros consecutivos empezando por **0**.
- Si se añade un elemento sin especificar la clave, la clave será el número siguiente a la mayor clave entera presente en el array.
- A la hora de comparar el “===” compara todos los elementos de un array en clave, valor, posición. Para el operador “==” el orden no importa.
- El operador “+” entre dos arrays devuelve la unión de ambos. Sin repetir claves. En caso de claves repetidas, no se añade.



# Tipos de datos compuestos.

## Funciones.

---

- Una función es un conjunto de instrucciones que realiza una tarea concreta.
- Las funciones pueden recibir argumentos.
- Las funciones pueden o no devolver un valor, no es necesario declarar el tipo de dato que devuelve.
- Existen dos tipos de funciones en PHP:
  1. Funciones predefinidas
  2. Funciones definidas por el usuario



# Tipos de datos compuestos.

## Funciones predefinidas

---

- En PHP se usan muchas funciones predefinidas para las tareas más habituales, como por ejemplo en tratamiento de las variables. Algunos ejemplos son los siguientes:
  - `is_null($var)`: ¿es nulo \$var?
  - `isset($var)`: ¿\$var ha sido inicializada y su valor no es nulo?
  - `unset($var)`: Elimina la variable \$var.
  - `empty($var)`: ¿\$var no ha sido inicializada o su valor es falso?
  - `is_int($var)`: ¿Es \$var entero?



# Tipos de datos compuestos.

## Funciones definidas por el usuario.

---

Para definir una función la sintaxis es:

```
function nombre (argumentos) {  
    <instrucciones>  
}
```



# Tipos de datos compuestos.

## Funciones definidas por el usuario.

---

Ejemplo de una función:

```
function suma($a, $b) {  
    return $a + $b;  
}  
  
echo suma(4,8) . '<br>';  
$var1 = 35;  
$var2 = 5;  
$var3 = suma($var1, $var2);  
echo $var3. '<br>';
```



# Tipos de datos compuestos.

## Funciones definidas por el usuario.

---

Es posible modificar valores por defecto para los argumentos. Si al llamar a la función no se usa el argumento, se toma el valor por defecto:

```
function saludar($nombre = 'usuario'){  
    echo "Hola $nombre <br>";  
}
```

saludar()      Hola usuario

saludar(Ana)    Hola Ana



# Tipos de datos compuestos.

## Funciones definidas por el usuario.

---

**Ejercicio:** Escribir una función para recibir potencias. Recibirá como argumentos la base y el exponente, que es opcional y que tiene por valor por defecto 2 (elevar al cuadrado).



# Tipos de datos compuestos.

## Funciones definidas por el usuario.

---

El paso de argumentos puede ser:

```
$var1 = 5;  
duplicarMal($var1);  
echo "$var1 <br>";  
$var1 = duplicar($var1);  
echo "$var1 <br>";  
duplicar2($var1);  
echo "$var1 <br>";
```

| Por copia                                                                                                                                | Por valor                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <pre>function duplicarMal(\$a){<br/>    \$a = \$a * 2;<br/>}<br/><br/>function duplicar(\$a){<br/>    <b>return</b> \$a * 2;<br/>}</pre> | <pre>function duplicar2(&amp;\$a){<br/>    \$a = \$a * 2;<br/>}</pre> |



# Tipos de datos compuestos.

## Formularios HTML.

---

Los formularios HTML son la forma más habitual de enviar datos a un servidor. Permite que un usuario rellene varios campos mediante diferentes tipos de controladores y lo envíe al servidor al pulsar el botón submit.

El servidor procesa los datos del formulario y genera la respuesta.



# Tipos de datos compuestos.

## Formularios HTML.

---

```
<!DOCTYPE html>
<html>
  <head>
    <title>Formulario de login</title>
    <meta charset = "UTF-8">
  </head>
  <body>
    <form action = "login_basico.php" method = "POST">
      <input name = "usuario" type = "text">
      <input name = "clave" type = "password">
      <input type = "submit">
    </form>
  </body>
</html>
```

---

```
<?php
  echo "Usuario introducido: " . $_POST['usuario']. "<br>";
  echo "Clave introducida: " . $_POST['clave'];
?>
```

**login\_básico.php**



# Tipos de datos compuestos.

## Formularios HTML.

---

En ocasiones el formulario HTML y el bloque PHP que lo procesa se integran en un solo fichero.

En esta situación hay que distinguir dos casos: Cuando el formulario es accedido para rellenarse y cuando se envían los datos para su procesamiento.

Si se accede al formulario mediante GET quiere decir que se accede para rellenarse pero cuando se accede mediante POST lo que indica es que ha sido rellenado (`$_SERVER['METHOD_REQUEST']`).



# Tipos de datos compuestos.

## Formularios HTML.

---

Veremos el ejemplo de un fichero **form\_en\_uno.php** que une los dos ficheros vistos anteriormente en uno sólo y, en caso de error:

- Muestra un mensaje apropiado encima del formulario.
- Mantiene el valor introducido en el campo usuario.



# Tipos de datos compuestos.

## Formularios HTML.

---

Un caso especial en los formularios, es la subida de ficheros al servidor. En el formulario hay que usar el atributo:

`enctype="multipart/form-data"`

Y el método **POST**.

Para el fichero se usa la etiqueta

`<input type = "file">`

Y en PHP se recibe con la variable global **\$\_FILES** que contiene información sobre el fichero que se está subiendo.



# Tipos de datos compuestos.

## Formularios HTML.

---

Veremos el ejemplo de un fichero **procesar\_subida.php** que se encarga de comprobar que el fichero no pase de cierto límite de tamaño (el directorio “subidos” debe existir previamente).

Las claves de **\$\_FILES**:

- **name:** Nombre del fichero en el cliente.
- **size:** el tamaño del fichero en bytes.
- **type:** El tipo MIME del fichero.
- **tmp\_name:** El nombre temporal con el que se ha subido al servidor.
- **error:** código de error asociado a la subida.



# Tipos de datos compuestos.

## Formularios HTML.

---

Como HTTP es un protocolo sin estado, las diferentes peticiones de un cliente al servidor son independientes, no están relacionadas entre sí. Para asociarlas, se utilizan las sesiones.

Al iniciar una sesión, el servidor asigna y envía al usuario un identificador de sesión. En las siguientes peticiones el usuario envía al servidor ese identificador, de manera que el servidor sabe que se trata del mismo usuario.

Para controlar la sesión el servidor deja una cookie con el id en la sesión del cliente.

Para crear una sesión se utiliza la función **start\_session()**. Si no hay una sesión activa la crea, si la hay, el script que llama a la función se une a ella.

Una vez creada la sesión, es posible utilizar **\$\_SESSION**



# Tipos de datos compuestos.

## Formularios HTML.

---

A continuación, se verá un ejemplo de control de sesiones que sigue el siguiente flujo:

